

◆ Cursors

✅ What They Are:

A **cursor** is a database object used to **retrieve rows one at a time** from a query result set — especially useful when processing large results or doing row-by-row operations in stored procedures.

✅ Typical Use Cases:

- Iterating through query results in loops.
- Performing complex logic or conditional updates on each row.
- Batch processing data in stored procedures.

✅ Example:

```
DO $$
DECLARE
    emp_record RECORD;
    emp_cursor CURSOR FOR SELECT id, name FROM employees;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Employee: %, %', emp_record.id, emp_record.name;
    END LOOP;
    CLOSE emp_cursor;
END $$;
```

❌ Limitations:

- Slower than set-based operations.
- Increased memory usage and complexity.
- Should be used only when row-by-row processing is necessary.

◆ Indexes

✅ What They Are:

An **index** is a data structure that improves the **speed of data retrieval** operations at the cost of additional storage and slower writes (INSERT/UPDATE/DELETE).

✓ Typical Use Cases:

- Speeding up search conditions (WHERE, JOIN, ORDER BY).
- Enforcing uniqueness (with UNIQUE indexes).
- Supporting text search (GIN/GiST indexes).

✓ Example:

```
CREATE INDEX idx_employee_name ON employees(name);
```

✗ Limitations:

- Increase storage size.
 - Can slow down data modification (INSERT, UPDATE, DELETE).
 - Too many indexes can hurt performance rather than help.
-

◆ Transactions

✓ What They Are:

A **transaction** is a group of one or more SQL statements that are executed as a single unit of work. It ensures **that all lines of code will execute completely or none of them will be executed**.

✓ Typical Use Cases:

- Ensuring data integrity in critical operations.
- Rolling back changes on error.
- Batching multiple operations safely.

✓ Example:

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

 --Or use rollback on failure:

```
BEGIN;  
UPDATE users SET email = NULL WHERE id = 5;  
-- some error check  
ROLLBACK;
```

✗ Limitations:

- Long-running transactions can lock resources and impact concurrency.
 - Improper handling can lead to deadlocks or inconsistent data.
-

◆ Views

✓ What They Are:

A **view** is a virtual table based on the result of an SQL query. It does not store data itself but provides a **logical abstraction** over base tables.

✓ Typical Use Cases:

- Hiding complexity of joins or subqueries.
- Providing a simplified or secure version of data.
- Enforcing row/column level access control.

✓ Example:

```
CREATE VIEW active_customers AS
SELECT id, name FROM customers WHERE is_active = TRUE;
```

✗ Limitations:

- Complex views are often **not updatable**.
 - Performance may be worse than querying base tables directly.
 - No indexes can be created on views (but **materialized views** can be indexed).
-

◆ Summary Table

Feature	Purpose	Can Modify Data?	Stored in DB?	Key Limitation
Cursors	Row-by-row processing	✓ Yes	✗ No	Slower than set operations
Indexes	Speed up data retrieval	✗ No	✓ Yes	Slows down writes, uses storage
Transactions	Group actions atomically	✓ Yes	✗ No	Can lock resources or deadlock
Views	Virtual table from a SELECT query	✓ Sometimes	✗ No	Often not updatable or indexable